

ActInf ModelStream 010.2 ~ RxEnvironments.jl with Wouter Nuijten

ModelStream | RxEnvironments.jl with Wouter Nuijten | 58:45

all right hello it is May 16th 2024

we're in the RX and fur JL project with

some guests from bias lab and

participants and vzer will be sharing

with us about the RX environments. JL

package so thanks all for joining

looking forward to this presentation and

discussion go for

it all right thank you Daniel uh so my

name is V I am a second year PhD student

in bias lab uh you may know Bas laab

because we are the uh main authors of

the RX infer package a package for

reactive Beijing

inference and last year when I was

working with RX and fur um and doing

well reactive beijan inference I found

that there was a a lacking support for

environments in the same reactive

Paradigm uh and we were relying on

reinforcement learning uh packages such

as gymnasium or or reinforcement

learning

and I didn't really like it so I wrote

RX environments. JL a package for

reactive um environments which aims to

uh complement RX and fur in the

reactive um agent

development um ecosystem so if we look

at RX environments it's a very

lightweight boiler plate package it

contains a boiler plate code to make

something that I call a reactive

environment a reactive environment is

a an environment that consists of

entities where an entity can be an agent
but it can also be a component of an
environment or the entire environment
itself and what we have to describe is
how entities communicate with each other
so every entity has its own Mark of
blanket with which it can
communicate with other entities in the
environment so in that sense it's very
um related to active inference because
well
we have a very specific notion of the
interior of an entity and the exterior
of the entity and it's Mark of blanket
and I try to replicate that in ARX
environments so um to give a short demo
we can dive right into the code I have
prepared um the mountain car example in
the uh documentation of ARX environments
but just to Showcase to you that this
gives you um more
um flexibility to work with so we have
on the left hand side I have my code
because I'm going to manipulate this
environment uh in real time and on the
right hand side I have the environment
so I have my landscape I hope you guys
can see my cursor by the way uh I have
the landscape and in the middle I have
one agent one car a DOT representing a
car um what's nice about RX environments
is that I can send um actions to the
environment so my landscape the
environment is one entity the agent the
car is a different entity and I can send
a command that contains Engine Force on
a scale from minus one to one so I can
send uh minus one to The Entity and

you'll see that the car starts driving
up the hill it doesn't have a lot of
Engine Force and I can live send
different commands and you'll
see that the car responds accordingly
now what's nice about arcs environments
is that as I said uh you'll get native
uh multi-agent environments out of the
box so if I just add a different agent
Mountain car agent with some hyper
parameters to the
environment you'll see that there is now
a second dot at the bottom of the valley
a second car and similar to how this
first car can be
controlled we can control the second car
too 01 was
maybe uh this uh I should have maybe
given this car a bit less engine power
because it's clearly flying off the
screen um but you get the gist um that
we
can um well control both cars
simultaneously um and RX environments
handles all this uh this communication
natively reactively under the hood so my
idea is or my next steps for this
project are going to be to uh
incorporate better into the ARX and fur
ecosystem because I think that building
agents uh with ARs environments in this
way U that both the active inference
Community as well as the arcs Ander
Community can can can benefit from this
so this is a very short overview of the
things that ARX environments can do it
can do a whole lot more um but I think
that this is a very short demo that

gives you the best idea of why it's nice
that we're going to go reactive because
you get stuff like this we can control
in real time we can control multi-agent
environments um and it's very um active
inference e in that sense so on that
note um I'll stop sharing my
screen let's see and um yeah if you guys
have any questions about this which I uh
which I heard you have then I'll be very
happy to take
it awesome
thanks very informative and hilarious
Mountain car example yeah this this you
you you can play with this Engine Force
of this mountain car and I probably
should have prepared that it doesn't fly
off the screen but you I mean it gets
the point for us right
it's yeah okay um I'll share screen and
just type and so everyone else in the
group just go for asking any of the
questions we'd already identified or any
new questions arising so go for
it um interested Rel
between I I I don't understand you very
well maybe there's do you understand c
as well okay
relationship
between you're you're very very
faint uh yeah okay
well with stuff I I actually had a
question do not on this particular page
that um Daniel's sharing right now I
tried my hand at let's say plugging in
the mountain car the existing Mountain
car agent in the RX and thir
documentation into your Mountain car

environment I wasn't quite able to do that but it appears that you you have done that uh in that example just then and then maybe I was missing something because in the in the documentation the RX environments documentation from mounting car I didn't see that there was a an agent that had you know instantiated the generative model uh much in the same way that the ARX inference documentation had and I thought oh okay this this is a wrapper or you know a nice API for me to specify an environment I I now have to go away and then specify my agent also but it seems is that what you did for that example just then or or did you just no no I just for this example just now I just used my um you know custom commands that I just uh sent myself um so this is something that I'm actively working on on uh combining the two um the mountain active inference Mountain car example on the ARX and fur website I believe is a bit um cheating in a sense because we give it all Environ Dynamics all functions of the environment Dynamics and then we uh do some uh some approximation to the posterior of what we have to do but in a sense it doesn't really discover that it has to do anything because it knows all the environment Dynamics so that's why I haven't really spent time on um incorporating this specific example because I feel that if we're going to demo this package plus an agent that it should be an agent that actually has

capabilities of exploring the environment instead of knowing all the environment Dynamics and just while basically doing model predictive control right okay that makes it a bit clearer okay I I have a follow up maybe let somebody else um something just for now thank you very much so to give a very short answer to your question we can we can sit together if we uh on if want to Define an agent a generative model I know how to hoop the two up I just don't have a working example yet right okay I have I don't want to use this I don't want to use this agent from from the arxs and F documentation because indeed I feel that it's a bit cheating but it's my personal opinion right yeah yeah okay that that actually clears a lot of things off for me because I was kind of unclear like oh do I do I kind of plug that one in is that is that what I do yeah okay but that's not that functionality exactly CLE yeah we can um however there is a a subtlety there and that is something that I find very interesting in my research is if you open up this space or you open up your environment to be reactive then you want your environment to be able to um receive an action at any point in time so what we see in reinforcement learning is that you have a specific transition function the environment Waits until you send an action it will move the environment forward a pre-specified amount of time and it will emit to you

an observation and a reward however if you make this entire thing reactive then well our agent would be able to emit at any point in time and also it would be able to get an observation at any point in time now that is a very interesting um setting to work with because now our agents should kind of be robust to getting an observation at any point in time specifically what if uh our agent that lives on some embedded device on some drone whatsoever it has less compute to do to do inference however well at some point we're going to have to send an observ or an agent an action back to the environment we're going to have to do something so this reactivity opens up uh a research Avenue for well right now for me because I find find this very interesting on how reactive truly reactive agents would look like if you see what I mean yeah yeah I do that's that's exceptionally exciting stuff it's it's awesome um okay but but okay just just to wrap up my my yes question there is no at the moment you kind of manually you know yes you put in the force limit and you say okay I'm going to kind of manually you know play around with letters and I have I have working examples with like ban therat or very very easy um which I probably I don't know if I have them on my machine right now or if they live somewhere like it works but the the thing that I got to work is model predictive control and not active inference yet so that's

why awesome all right thank

you you're welcome

um is my yeah can you hear me a bit

better now yes okay but um I'm uh

interested in the relationship between

orx environments and orex in fur um for

example to put it simply is it possible

to use only orx environments or is it

supposed to be always be used in

collaboration with RX and

fur

us yes the relation is that uh I wrote

ARX environment

specifically um as a complimentary

package to ARX and fur so if you just

want to do basian inference you use ARX

and fur but I think if you want to build

agents that you're better off using the

the two together so RX environments

is um a package that uses the same

reactive backend as RX and fur so it's

made to be compatible with ARX and fur

however if you write um agents just with

an algorithm you can use ARX

environments by itself it's not

mandatory to use ARX and fur they are

separate packages there's not a

dependency on ARX Ander however since um

they share the same reactive back endend

so the same package for reactivity under

the hood which is also from bias laab

which is rocket um if you want to build

agents I would advise to to use RX INF

fur so there they're disjoined but

they're designed with each other in mind

or ARX environments is designed with ARX

Ander in

mind so um if we um use this high level

of a
of abstraction given to us by ARX Vons
uh do we lose
anything uh or can we at any time dive
into the lower levels by means of orex
INF fur
itself
so um I would say yes because the
inference process that happens in ARX
and fur is disjointed from what's going
on in ARX environment similarly to well
what happens in in in real life so you
can you can see ARX environments as a
package in which you can make
um like a digital twin of the
environment but then whatever happens in
the inference process in your agents
well that's still RX andur like all RX
andur um diagnostic tools or debugging
tools should be available
there um and a more practical question I
know from based on the large papers um
and uh Professor def as well they like
to divide the um what's going on into
these six methods the ACT future execute
observe compute and slide so do you see
that as kind of Superfluous in the orx
orx environment setting
or um is it is it fine to use that
approach
let me Google the definition for
Superfluous uh ah um that is a I'm going
to spread very carefully in answering
that questions uh I don't think it is
Superfluous um because in a sense that
it's if you want to design your agent
such that it goes through those steps um
then you're more than welcome to uh my

personal opinion is that I'm I will I'm
going to try and see if I can make an
active inference agent that does
inference in one step instead of having
to act

observe uh plan slide uh and so forth
however for AR environment so for your
environment it doesn't really matter how
you will get to your uh to your action
as long as well or not right you can you
can send an action at any point in time
and if you don't send an action the
world will just continue that and arcs
environments will just continue running
there's no real constraints on that so
that the design is to be as less
constraining on on users as possible
while opening up this reactivity uh
um the reactivity aspect and also for
um if you start with the package you
could make what I call discreet
environments they're they're a bit
feeding I would say because they're
basically just the arc environment's
equivalent of a of a classical
reinforcement learning environment where
you have a prespecified time step every
time but I wanted to make that to make
the transition easier because if you go
full reactive right off the bat I would
say it's a very ambitious thing to
do so I don't think I don't think it's a
Superfluous
algorithm
um but I I also don't think it's a
um
well our environment doesn't constrain
you to use it so I would

say nice now let's say uh one has a need to write one's own node in the factory um put ARX environments support that kind of thing or is it kind of an irrelevant

question it's it's in that sense I would say irrelevant because uh the um a custom node is something that lives in ARX and for um and ARX environments really uh is concerned with everything that goes on outside of your inference process um then another quick question while playing with the um I think it was a thermostat problem in oric

environments I wanted to see the time behavior of some variables like observations States controls um free energy and things like that but it wasn't straightforward to me how to plot those um in the argument settings uh is there uh an example that you can point as to or a fragment of code that shows us how to do

that um I can make it because I I know how I can access so free energy is uh something that lives inside your agent so it's part of the inference process it's the you know it's the uh cost function if you will of of of the inference process so for free energy you would have to look at your uh would have to look into ARX and for however everything that goes on in our environments we can extract it and we can plot it so both uh environment States as well as emitted controls uh or observations uh we can extract all those things um I can show a code

fragment that does this because it's
well one line um however I also
understand that that is not something
that
uh well you probably remember after this
meeting so I'll I'll make sure that uh
we can design a
uh uh an example that showcases this but
in uh in a nutshell we can uh create a
keep actor so this is a well uh an
entity that will keep all everything
that it gets
uh we'll keep it inside memory and then
we subscribe to the observations of the
agent so we say well subscribe to
observations and with this Ops keep
actor uh we do
this and if I then call OBS
values this will contain all kinds of
well ARS environment
observations which is basically a
wrapper struct for uh let
me call data on this entire
array and you'll get your individual
observation so we can basically tap
anywhere in this process uh anywhere in
this communication we can we can tap
whatever is being emitted so if I run
this again now it will contain 400
elements because well this environment
is reactively running while we speak so
the the the agent is getting more and
more observations so you see this thing
reactively growing every time I call
uh or extract its uh its contents so
there is definitely functionality in
which you can sort of inspect this
process um but yeah I I I have to stress

that it's really distinct from the inference process so if you make an agent in ARX infer and you hook it up to ARX environments then you have to set up a similar procedure to extract free energy and stuff like that um from ARX and fur um and because I'm not the author of ARX and fur I I don't know how easy that is um but for AR environments I yeah this is one of the first things that I wanted to do see what my agent was sending see what my agent was observing so I made sure to build it yeah I'm not sure if what you just showed was uh recorded by us so if you could uh just make sure that that is included in your examples as well that would yes that that's my idea that we can uh or I I just wanted to Showcase uh for now such that you can so that you know that it exists but it wasn't meant to be well a big tutorial or anything I can just put it in the examples and uh uh yeah then then we can plot it that's that's not a problem

okay thanks a lot you're welcome okay anyone else can go for a question or we have some other written ones or I can ask a question um I have a quick question I would just wonder once I don't know if you guys can hear me I hope so yes um fantastic um a lot of the examples that uh you have there and kind of the way I'm I'm probably naive in how I'm thinking about this um the ARX environment is modeling something in the

real world um or at least that's the example that we're thinking of right now um is the is arc's environments capable of essentially modeling um more metaphysical spaces so if you think of your question in a way that you have a Time series or a space that you're trying to occupy but it's not a physical space it's a more theoretical space uh you know dimensionality that you're trying to have an agent navigate um is that something that this that you see that this is you know usable for or is this something that we need to kind of gear back towards the arent for um package itself to kind of build the more metaphysical concept of the environment that you're putting together does that make sense so yeah if I understand your question correctly um yeah I my answer to that would be that there's no a physics simulator or whatever in arcs environments it's purely the boiler plate to make whatever you write act reactively uh and expose reactive streams that you can then use in arcs and so um for example one of our colleagues has made a an environment that Returns the um the result of a stochastic differential equation so an agent could query with an action I want the realization at time T and the environment would respond with

the actual value u_m and with this interplay of query in a stochastic differential equation and u_h and getting a result you can cast solving a differential stochastic differential equation sort of a u_h aasan stat based model u_h I'm not an expert on this topic I just know that one of our colleagues u_h did this so if I understand your question correctly that would be the sort of environments you were after or you were you were questioning about u_m so I would say yes you can so there's no there's no physics simulator or anything in there because that's a whole different u_h a whole different topic u_m my main thing was just to make whatever code you guys write to make it reactive fantastic thank you now that makes a lot of sense I just wanted to clarify that since no it's perfect not thinking so much in robotics or querying like physical environments and thinking more along the lines of data environment so that's why I was trying to understand how we could utilize this yes yeah you can yeah and u_h you can push it further right you can make a u_h a combination of the two you can make a u_h an environment that flies a drone u_h your agent has to control the Drone but it can also query the environment for hey what is going to be my position in 5 seconds so kind of this this well abstract action that's basically querying the environment well since we have this u_h this mechanism that we can

depending on what action you send
correspond or match this with an
observation now you can do stuff like
that there's nobody stopping you doing
it whether or not it's useful for your
agent well that's up to you I I was just
yeah I'm I'm trying to make your
environments as expressive as can or
your the framework as expressive as I
can and yeah you you can you can go wild
with it right you can go
crazy that's perfect because the agent
that I'm kind of that I'm wanting to put
together would need to actually walk
through time in a certain way um and
optimize its activities in a Time series
um yeah and I was I was already
expecting you that's why I brought up
this this uh that's very cool I be very
excited to see uh well whether or not
you uh you can build your environment
and ARS environment would be yeah very
cool

Absolutely I'll keep you updated then
nice

thanks the observable and actor roles in
orx

environments can any entity take on
either of these roles or are there
certain constraints for example
let's say the agent must always have the
actor role or

something no they're completely
unconstrained um there is no real
distinction under the hood

between uh entities or between something
that represents an environment as
something that represents an agent uh

they are both entities in arcs
environments um the only thing is that
you can make uh something which I call
an active or a passive entity by the way
all this terminology it's all up for
discussion I've thought about this and
most of the terms that I come up with
they they don't really fit what I what
I'm trying to say um but an active
entity is something that whenever it
gets an observation or whenever it
receives an action it emits or it at
least considers emitting to uh
subscribers for example the mountain car
uh environment itself would be an active
entity whenever the the agent sends an
action to move the car it will
correspond or it will match this uh this
this action with well moving the car um
whereas the agent whenever it gets an
observation from the environment well it
doesn't really have to do anything we
have to in the later stage attach an
inference process but that is something
that's not ARX environments that is ARX
and first so we we make this entity um
passive in a sense that it's it's an
observer uh that can also send actions
but those are not triggered by an actual
algorithm that we write ourselves uh in
arxs environments but it's a result of
some inference process it's the triggers
are external to the the reactive
ecosystem that we've built in ARX
environments
itself so that's the only distinction
that I make but you can yeah something
that represents an environment or

something that represents an agent
they're they're under the hood
completely the same things they are just
uh entities that that that can
communicate with with with external
entities do you think you could um in
addition add a a discrete control space
example for us as well because Mountain
car and the evasion thermostat are both
um continuous
examples even if it's just a simple uh
discrete control space or categorical in
particular categorical control
space maybe a simple maze example or a
grid example you know like the windy
grid that's often used in reinforcement
learning even if it's just a simple
windy
grid windy grid yeah okay I'll I'll
Google
that just uh even if it's just a simple
categorical control space because that's
what our problems currently uh center
around not so much continuous control
spaces but uh categorical control
spaces okay that's uh yeah I can do that
okay I
windy G world yeah okay I I can do
that be
excellent I ask sort of a very quick
broad question about the whole
ontology my when I first learned about
this my own Na and and just sort of
looking through the docs I thought oh
okay this is this is a rapper around RX
and thir for standardizing the creation
of an Environ enironment that you will
then use for for whatever purposes but

it sounds to me it's it's not at all
that it's it's really this is this is
not even really a way of talking about
an environment per se this is no it's
more of a it's a communication protocol
for active inference agents I would say
yeah yeah it's like well you this this
is a way to sort of standardize the the
diatic interaction between you know two
abstract entities U and that could be
between environment and an agent you
know or it could be something else
entirely okay that's much much broader
than what I was conceptualizing ex so I
I when I when I started this indeed I I
started as okay I want to standardize
environment Creation in in our ecosystem
in our fer and I had discussion with
Magnus kudal who's also uh in in this
Discord I saw and well we had some very
interesting talks and he said well
actually an environment and and an agent
they're not so different right they
just they just hand or they just receive
data from each other and and handle them
accordingly so yeah there is a certain
uh symmetry there and then I thought
well if there's a symmetry there then I
can make an
overarching uh programming concept and
just kind of implement it that way and
that's what I did and then it turns from
just a way in which you can make an
agent and an environment communicate to
a in which you can communicate this
whole network
of uh of subscriptions and
Communications um because there's yeah

there's no constraints anymore so for example we run an environment in in uh in our lab with MCO actually that um has well a non-trivial uh entity interaction where we have a hearing aid that uh gets data from both the external world because it can well it has a microphone and it should register the external world but it can also get data from the user of the hearing aid saying hey I'm happy with the hearing aid settings yes or no so now we have a three entity environment we have the user we have the hearing aid and we have the uh the external world that just emits noise then we have a fourth entity which is our agent that should learn the preferences of the user that should get data from the hearing aid and should uh tweak the parameters inside of the hearing a so now we have four entities that are communicating in this non-trivial way and well we built this in arcs environments and it all holds up like it all just yeah it runs just because whenever we get data the hearing aid knows ah I get new data I get new noise I have to transform this noise and send it to the user and I have to do a f8 transform and send it to the agent for analysis uh and just because it does this reactively it's all tractable and all works nicely so yeah uh it it it turned out to be a bit of a uh a bigger well scope than I would have imagined but I'm yeah I I'm I'm not making any claims on on how large it

actually is that's uh yeah

cool this this is also very cool I'm

very very thankful that you're you're

working on this because I I I had the

thought like oh what what what then is

you know sort of predicated of the

environment it's kind of like what we

have the environment and the Asian

they're kind of both being equally

treated

you it's a very interesting question

right so so what the what is the actual

actual

difference um some people say well an

agent has some sort of inferential

process uh on how it comes up with an

action but in a sense well we are all

all just with our eyes and ears and

muscles we have an interface to to the

environment um but yeah in in in that

sense we we are just all Mark of

blankets with internal states that that

that navigate this world uh and this

well communication that we have with our

outside world I yeah I try to replicate

fastic

thanks anyone else want to ask any

questions yeah Kristoff or yeah yeah I

have a question um I I didn't quite get

what you um what you said about the

difference between the typical

reinforcement learning setting and the

RX environments Could you um could you

could you explain that again please that

was very interesting sure sure if if you

want or if you make an environment in JY

in gymnasium we can we can look at

the um at their documentation right so

let's see

gymnasium

package so I go to gymnasium and I want

to make my own custom

environment I saw it here make your own

custom environment i' I've visited this

page before so we can make our

environment blah blah blah and then you

have this function step the step method

usually contains most of the logic for

your environment it accepts an action

computes the state of the environment

after applying that action and returns a

five Tuple observation reward terminated

truncated

info so we write a function step that

takes only an action so this is the main

design Paradigm of JY and most

reinforcement

learning packages actually where I emit

an

action this action is incorporated into

the world um but the time that uh the

environment is uh simulated forward is

constant so there's no notion of

different time intervals whereas if I'm

I'm a very big uh football effici out

for the Americans that's soccer um but

it's actually football but if I kick a

ball I kick the ball the ball starts

rolling

well whether or not I do something the

ball will just continue rolling like

this this environment around me it

doesn't wait for me to emit additional

actions or I'm not consciously saying

okay for the next second I do nothing

and then the ball rolls okay for the

next second I do nothing again and the ball rolls further no there is a in the real world there is a difference right we do something and then the world just continues

so we can do something at any point in time and the world around us will just well the world keeps on turn um and that's what I was trying to do with with ARX environments is to um alleviate ourselves of

this um communication protocol because this specifies I get one

observation I can do one action per observation and that's it and in between uh different observations I have no power to do anything whatsoever also if I have an environment that instead of modeling one second forward I now do half a second well everything breaks down because now I have to emit actions at twice a speed or my observations don't correspond anymore with my internal model or there's all kinds of stuff that if you do it like this it turns kind of awkward and that's what I was trying to

alleviate

uh with with ARX environments is that I'm trying to make this communication between agent and environment as free as possible so if you have a hearing aid environment yeah that you can yeah that's that that's it if I'm not yeah

I'm rambling sorry good interrupt that's a good that's a good explanation and U I think you also mentioned something about um some difficulty there because uh the

the question then becomes like if you have multiple agents in with the environment then what exactly is the state of the world right um what exactly is the state of the world is not uh because you can just uh sort of you can have like a god entity that takes care of of of this stuff that that that kind of resolves all conflicts in that sense um the main question for me would be well we just took away your one moment to conduct an action in reinforcement learning you have one moment you can do an action and you'll get one observation we just took away this one moment and gave you the opportunity to act at any point in time well when are you going to do it right so you know the reason I'm asking about this is that it it immediately got me thinking about um stepen wolfram's work on Observer Theory so he's trying to formalize uh the notion of an observer right and um there is this um whole field that he basically invented uh for the physics project related to uh which calls multi multi computation I'm not really deep into um that yet but it seems some it seems somehow relevant you know as an agents interacting with the environment we effectively sample it precisely because it keeps rolling as we um do the internal computation required to um update our um our generative model in order to to act so I I think you stumbled onto something quite quite

profound there um and the question the question is about this this God God mode view right like that's a that's an old question in physics whether there exists the absolute reference frame or whether everything is relative um and yeah just won't say that you know foot for fault there's there's some there's some interesting problems there maybe we can we can talk about it some sometime I would love to hear your thoughts about this yeah sure like I'm I'm very interested in this stuff so as a as a software developer like I would advise you in your environments if you're not dealing with you know existential stuff that just make a god state that takes care of this but uh you make a very good point right so if we really want to make an accurate representation of of the world do we need this God state or because we now allow our environments to to alleviate this right we can have multiple entities that together compose the environment and that there is some internal communication between them so um it does it does open up this like it doesn't slam it wide open but at least you know there's a in Dutch you would say open key yeah so so I I think there is a there's a that that's that's a potential um incursion into effectively resolving some of the most interesting problems in the fundamental theories of physics that understanding precisely which one is the case whether is this um absolute

reference frame or whether you can just simply explain everything through these relative reference frames and agent sampling the environments and if if you can effectively recover something that looks like physics from um this interaction which is what wall from is essentially working on that would be that would be a complete computation of fundamental Theory of physics and it's a bit big undertaking but it's worth uh worth considering not just merely from you know um software engineering perspective but from from that angle if if you're interested in that kind of stuff I mean yeah it it's very interesting it's something that I you you know it's a very big deviation from what my supervisor wants me to work on so it's probably not something that I'm going to undertake but yeah if if anyone wants to pick it up right who am I to stop

them thanks it's great work

cool yeah anyone else want to ask any questions

how would you deal with the setting of multiple ant nestmates moving around and modifying the pheromones on the ground so in these kinds of simulations it's common enough to have the pherone phermone state on the ground and then Nest mates that are reading and writing how do you deal with the spatial proximity and the collisions and interactions

so that is something very very very difficult I'm working on a project right

now where once again we're modeling
football uh which is why I brought up
this example just now um and like
resolving collisions is something
extremely difficult it's something that
I don't have a standardized uh advised
way of doing an arcs environments just
because game studios uh they they spend
God knows how many developer hours on
resolving collisions efficiently so
that's not something that I think well
we we can figure out in an afternoon
right
um it's it's it's something that would
be in the internal dynamics of your
environment but not something that is
part of your communication protocol per
se just influences the the the type of
observations that your agent would get
um so it's not something that I can give
that I feel qualified to give advice
on but it is like making this
environment with ants running around
the floor that is something that's just
a multi-agent environment that's fully
covered the problem is these these uh
the collisions that is a difficult thing
it's not uh difficult in an arcs
environment uh perspective it's
difficult in a computational geometry
perspective that you want to resolve
collisions that's yeah like in in tripa
a games you can still clip into walls
somewhere
that's that's a great and funny response
yeah I would just highlight the the real
structural difference when you pointed
to the step function in the

gymnasium that yes is why in so often for these control problems there's like a null action or like a weight action so you kind of get this um bodal action Vector policy Vector where there's like a do nothing option and then all the options because out of requirement something must be emitted at every click of the simulation and all of a sudden it's like we mve from walking on a staircase to like ice skating or flying with RX and fur because it has totally opened up a dimension of the control question when to act not just which with according to a metronome and that's that's an entirely different approach that um the communication protocols enable and then those communication protocols can have all these other functionalities like they can have a subontology or they can have compliance features as well so it's super powerful and we're very excited to learn it that's yeah that's the main point uh the yeah it's a two sides of the same coin like it it opens up this communication protocol but then yeah you have to question when when it's something something that I'm going to spend time on in my next project like can we make an iterative procedure maybe with iterations and ARs and third we can send intermediate posteriors and we have we always have like a an action that we Supply I I don't know yet I have no idea or I have ideas but I don't have anything working yet I think to me the underlying

principle of the RX in first Suite
versus
traditional yes this um
um instead of having synchronized
Communications we now have asynchronized
communications between everything
between everything
yes I'm I'm glad I got that point across
that's that's the point I wanted to
make great well um if anyone else has a
question otherwise we will continue
learning and watching the GitHub updates
and just building out examples CU in the
coming months we know a lot of fun
things will
happen all right yeah um you guys can
always reach out to me on I don't know
GitHub Twitter I don't know how often
I'll check Discord I'll try you know
I'll try um but you can always reach out
to me on uh on LinkedIn Twitter whatever
that's that's all fine
excellent I I do have a a final question
if that's all right um sure just just
with respect to you know future
directions and that kind of thing what's
kind of biggest problem set of problems
what are you kind of looking to do I
know that's a massive thing but is there
anything relatively summited in that
direction that that we could have speak
to for just a second so I'm very
interested in this when to act uh
problem so this to me opens up the door
to a um an agent
that um because we okay so we can now
act at any point in time or at any point
in time um yeah we can send something to

the environment a logical consequence for me is that at any point in time we should have an action ready to send so when the next observation comes in we don't know when that's going to be we should be able to act so we have a certain computational budget between one observation and the next we don't know how much it's going to be but we do have a budget to spend so I want to design indeed an interactive procedure that comes up with an action and then refines it and refines it and refines it until well the next observation comes in so we have to send whatever we had and we get the the next observation this gives you if this all works right this gives you a very nice robust agent that if you give it a lot of computing power a lot of computational budget then in between observations well it's going to be able to come up with a pretty good um pretty good action and will solve the task accordingly like ideally however if you limit this uh uh this computational budget you put it on like a Raspberry Pi or on an edge device it will not have as much iterations to refine its actions so it will still solve the problem hopefully but it will be less efficient doing so it will be slightly Dum so this is a very to me is a very interesting problem to think about because then you have a sort of a a procedure for control that is robust to loss of computation or whatever and it's still adaptive and reactive and and and all those nice things we get from from Asian

inference um so yeah that's something that I that I would want to work on if I you know at 48 hours in a day it's very interesting so it sounds to me like a number of iterations you just refer to as um the iteration field in the call of the infer function yes in infer yes if I can do something like that so I run an indefinite amount of iterations and I just stop the inference procedure whenever I have to do something and then I extract the latest posterior for the action and I I send that to the environment yeah that's something well that would be the first thing I would investigate yeah I I think it has the potential to make the behavior in general quite robust if you if you consider that approach it yeah it would be rob it would always have something to do do like it would always do something whether or not sort of the the the free energy would converge uh or yeah as as a measure of whether or not this action is good enough uh that is to be investigated but it would allow us to run the same models on edge devices as well and get slightly worse performance but still yeah yeah it would still do the job we and then you can investigate you know how many iterations do I actually need before I can solve the problem uh it's of course problem dependent but yeah it's a very yeah it's just something that that really interest me I

really like this this way of thinking um
and it's in part why I why I wrote this
package because I I don't like this this
this synchronized communication protocol
from Jim I just I just don't like it
yeah that's why I wrote wrote ARS
environments I think the only
alternative is to not be able to respond
whenever it gets an action other words to
postpone your
response it NES back into the um the the
synchronicity approach again so it
breaks away from the you know the
reactiveness of
the but then you you run into the
robustness issue I would say if you have
a self-driving car and it it's driving
into a wall because it's thinks it's a
green light because those things are not
that that smart nowadays uh and ah it
figures out it's a wall but it takes
some time to adjust or it takes some
time to come up with the idea okay now I
have fully converged on the action that
I have to steer away well by that time
we might have already crashed into the
wall um so this this queuing up of uh uh
of observations and waiting until uh my
computation is fully converged I think
it takes away the robustness like if you
if if you don't have enough
computational budget to
spend well yeah you you might do the
wrong thing it's the same as when you
and I are in a car right if we're in a
very
um a very busy situation so well I'm in
the Netherlands so we have cyclist right

so if we have cyclists we have other cars and there's an ambulance coming what it's very busy you might feel overwhelmed but you still kind of know the basic things to do to to to steer your car it's basically the same where you have limited computational budget there's all this stuff around you so to come up with a really thought through response or action uh it's too much to ask because you don't have the computational budget but your brain kind of prioritizes and says well if we stay on the road and we let the ambulance pass and well we're fine for now we're this is the best we can do um so this feeling of being overwhelmed by what what goes on around you inspired this idea of robustness and uh uh of interruptibility basically yeah but then again how how how I'm going to do it well no idea yet or run our inner iterations and see if I can uh can send intermediate posterior results or something or but it's just something yeah I find this very very very interesting think it's a good thing that free energy in general decays uh predictably in a sense so I mean just fully using your budget your computational budget for that time that you're allowed before the next observation arrives I think it's fairly safe to just uh yes use the current value of the posterior that's the best you can do the circumstances yes that that that would

be the argument

indeed hopefully hopefully this this

this opens up the road to having actual

you know robots driving around instead

because

well there's no self-driving cars where

I live there's no robots or drones

flying around and I think one of the

reasons is because they're not robust or

interruptible

fly agreed

yeah all right thank you all we'll look

forward to staying updated

asynchronously even asynchronously

reactively see y all soon

bye thanks